Inference in the log domain

Simon Frost

Table of contents

1 Overview 1

2 Setup 2

3 A model whose evidence underflows 2

4 Where ordinary arithmetic gives out 3

5 Evidence mass, and telling rare from impossible 4

6 Every marginal at once 5

7 What this does not promise 6

8 Summary 7

9 References 7

1 Overview

Every backend of the previous vignettes multiplies probabilities together. On a small network that is harmless, but the joint probability of a long run of observations is a product of many numbers below one, and Float64 runs out of exponent at about 10^{-308} — and out of *precision* well before that. When the evidence mass underflows to zero, an exact backend cannot form a posterior at all: dividing by zero mass is not a rounding error, it is a missing answer.

The awkward part is that a genuinely impossible observation also has zero mass. In ordinary arithmetic the two are indistinguishable.

This vignette covers the log-domain layer, which addresses both:

- `LogVariableElimination` and `LogJunctionTree` carry centered log-domain factors and combine them with log-sum-exp, so the intermediate quantities stay representable.

- `log_evidence_probability` returns the log evidence mass, which is an ordinary finite number for evidence that is merely rare, and `-Inf` for evidence that is impossible.
- `LogInferenceDiagnostics.mass_status` reports `:finite`, `:underflow` or `:zero`, so a run says which situation it was in.

The default backends do not use this layer themselves. When their Float64 evidence mass is not a normal positive number, they recompute the answer in exact arithmetic and round each cell once (ADR 0014, ADR 0016). The log domain is the fast opt-in alternative, and the last section is explicit about what it does *not* promise.

2 Setup

```
using BayesianNetworkInference
using FiniteKernels
using BayesianNetworks
```

3 A model whose evidence underflows

A chain of repeated detections is the simplest way to get a very small but perfectly ordinary evidence probability. `D1` is rarely positive, and each subsequent site copies the previous one with a little noise:

```
function detection_chain(n)
    vars = [Symbol("D", i) for i in 1:n]
    bn = bayesnet([v ⇒ [:no, :yes] for v in vars]...;
                  mechanisms = vcat([vars[1] ⇒ ()],
                                    [vars[i] ⇒ (vars[i - 1],) for i in 2:n]))
    m = bind_cpt(BayesModel(bn), vars[1] ⇒ [0.999, 0.001])
    for i in 2:n
        m = bind_cpt(m, vars[i] ⇒ [0.999 0.001; 0.9 0.1])
    end
    return m, vars
end

m, vars = detection_chain(5)
length(vars)
```

```
5
```

Observing `:yes` at every site but the last is legitimate evidence: it just becomes less and less likely as the chain grows. The chain is Markov, so the answer is known in closed form and does not depend on the length at all — conditioning on the previous site leaves the last one depending on nothing else, so the posterior is the second row of the transition table:

```
exact = [0.9, 0.1]
```

```
2-element Vector{Float64}:
 0.9
 0.1
```

That closed form is the oracle for everything below.

4 Where ordinary arithmetic gives out

The Float64 evidence mass goes through three regimes as the chain grows: a normal number, a subnormal one, and zero. The log-domain backend reports the log mass and its status alongside the default backend's answer and whether it fell back:

```
function compare(n)
    mm, vv = detection_chain(n)
    ev = Dict(v => :yes for v in vv[1:(n - 1)])
    q, info = infer(mm, vv[n]; evidence = ev)
    p, d = infer(mm, vv[n]; evidence = ev, backend = LogVariableElimination())
    return (n = n, default = round.(q.table; digits = 4),
            route = info.exact_fallback ? :exact_fallback : :float64,
            log_domain = round.(p.table; digits = 4),
            log_mass = round(d.log_evidence_probability; digits = 1),
            status = d.mass_status)
end

[compare(n) for n in (300, 320, 340, 400)]
```

```
4-element Vector{@NamedTuple{n::Int64, default::Vector{Float64}, route::Symbol,
log_domain::Vector{Float64}, log_mass::Float64, status::Symbol}}:
 (n = 300, default = [0.9, 0.1], route = :float64, log_domain = [0.9, 0.1], log_mass
 = -693.1, status = :finite)
 (n = 320, default = [0.9, 0.1], route = :exact_fallback, log_domain = [0.9, 0.1],
 log_mass = -739.1, status = :finite)
 (n = 340, default = [0.9, 0.1], route = :exact_fallback, log_domain = [0.9, 0.1],
 log_mass = -785.2, status = :underflow)
 (n = 400, default = [0.9, 0.1], route = :exact_fallback, log_domain = [0.9, 0.1],
 log_mass = -923.3, status = :underflow)
```

At 300 sites the mass is an ordinary Float64 and the default backend answers directly. At 320 the mass is subnormal: it is not zero (so `mass_status` is still `:finite`), but most of its significand has gone. At 340 it underflows to zero, which Float64 cannot tell apart from a contradiction. The default backend trusts a mass only when it is a normal positive number; in the other two regimes it reruns the query in exact arithmetic and says so in the diagnostics' `exact_fallback`.

What the subnormal mass would have cost is worth seeing, since the backend declines to pay it. An empty query returns the mass as Float64 computed it, with no exact fallback (ADR 0011), so

putting the query variable into the evidence recovers the two cells the backend would otherwise have divided, and they can be normalised by hand:

```
let
    n = 320
    mm, vv = detection_chain(n)
    ev = Dict(v => :yes for v in vv[1:(n - 1)])
    mass = only(first(infer(mm, Symbol[]; evidence = ev)).table)
    cells = [only(first(infer(mm, Symbol[];
                              evidence = merge(ev, Dict(vv[n] => s)))).table)
             for s in (:no, :yes)]
    (mass = mass, subnormal = issubnormal(mass),
     normalised_by_it = round.(cells ./ sum(cells); digits = 4))
end
```

```
(mass = 1.0e-321, subnormal = true, normalised_by_it = [0.901, 0.099])
```

That is the answer the default backend would have returned had it trusted the mass. Both paths it does take return the exact answer throughout:

```
all(compare(n).default == exact && compare(n).log_domain == exact
    for n in (300, 320, 340, 400))
```

```
true
```

To four decimals, that is. The two paths are not the same computation. The exact fallback returns each cell as the Float64 nearest the exact posterior of the model as bound; the log domain takes logarithms and exponentials on the way, and each of those rounds:

```
m340, v340 = detection_chain(340)
rare340 = Dict(v => :yes for v in v340[1:339])
(fallback = first(infer(m340, :D340; evidence = rare340)).table,
 log_domain = first(infer(m340, :D340; evidence = rare340,
                          backend = LogVariableElimination())).table)
```

```
(fallback = [0.9, 0.1], log_domain = [0.8999999999999999, 0.09999999999999998])
```

The middle regime is why the test is “normal”, not “nonzero”: silent precision loss arrives *before* outright underflow, and only the closed form would reveal it.

5 Evidence mass, and telling rare from impossible

`log_evidence_probability` reports the mass directly, without forming a posterior:

```
m340, v340 = detection_chain(340)
rare = Dict(v => :yes for v in v340[1:339])
log_evidence_probability(m340; evidence = rare)
```

```
-785.181516710967
```

A finite number, far below `floatmin(Float64)` but entirely well defined. Contrast it with evidence that genuinely cannot happen, here forced by a deterministic mechanism:

```
det = bayesnet(:A => [:no, :yes], :B => [:no, :yes], :C => [:lo, :hi];
               mechanisms = [:A => (), :B => (:A,), :C => (:A,)])
dm = bind_cpt(BayesModel(det),
              [:A => [0.5, 0.5], :B => [1.0 0.0; 0.0 1.0], :C => [0.5 0.5; 0.2
0.8]])
impossible = Dict(:A => :yes, :B => :no)
log_evidence_probability(dm; evidence = impossible)
```

```
-Inf
```

`-Inf`, not a small number. The two cases are distinguishable here, which they are not in ordinary arithmetic, where both produce a mass of zero. That is what lets the default backend answer one and reject the other:

```
(rare_chain = round.(first(infer(m340, :D340; evidence = rare)).table; digits = 4),
 contradiction = try
     infer(dm, :C; evidence = impossible)
 catch e
     typeof(e)
 end)
```

```
(rare_chain = [0.9, 0.1], contradiction = ImpossibleEvidenceError)
```

`ImpossibleEvidenceError` therefore means probability exactly zero: the answer does not exist. `mass_status` records which situation a log-domain run was in:

```
(rare = compare(340).status, ordinary = compare(20).status)
```

```
(rare = :underflow, ordinary = :finite)
```

An impossible observation raises under every backend, since there is no posterior to return, but `log_evidence_probability` gives `-Inf` rather than an exception, so a caller can test feasibility before committing to a query.

6 Every marginal at once

`LogJunctionTree` applies the same arithmetic to the Shafer-Shenoy collect/distribute schedule of the previous vignette (Shafer and Shenoy 1990), so one calibration answers every variable:

```
posteriors = all_marginals(m340; evidence = rare, backend = LogJunctionTree())
length(posteriors), round.(posteriors[:D340].table; digits = 4)
```

```
(340, [0.9, 0.1])
```

`log_calibrate` exposes the calibrated tree itself, with the log evidence and the centered component beliefs, for callers that want the intermediate quantities rather than the marginals:

```
calibrated = log_calibrate(compile(m340); evidence = rare)
typeof(calibrated)
```

```
LogCalibratedJunctionTree
```

A query spanning several cliques falls back to log variable elimination and says so in its diagnostics, exactly as the ordinary junction tree does.

7 What this does not promise

The backend returns Float64 posterior *cells*. Keeping the intermediate mass in the log domain prevents the intermediate product from underflowing; it does not make an unrepresentably small answer representable. A posterior cell below `floatmin` still underflows on the way out, and this is not a general error bound:

```
floatmin(Float64), exp(-785.2)
```

```
(2.2250738585072014e-308, 0.0)
```

Two further limits, both deliberate. A factor graph rejects an entry that is not finite or is negative by more than its tolerance when it is built (`FactorEntryError`), exactly as a kernel does. An entry a hair below zero — the kind rounding produces, within the tolerance — is a valid entry, but it has no logarithm, so the log backends reject it rather than silently producing `NaN`. The default backend accepts it, but a posterior cell that comes out negative is not a probability, and it refuses to return one rather than clamp it (`IndeterminatePosteriorError`, ADR 0014):

```
X = FiniteAxis(:X, [:a, :b])
rounded = FactorGraph([Factor(X, [1.0 + 1e-12, -1e-12])])
outcome(f) = try
    f()
catch e
    typeof(e)
end
(invalid = outcome(() -> FactorGraph([Factor(X, [0.5, -0.5])])),
 log_domain = outcome(() -> infer(rounded, [:X]; backend =
LogVariableElimination())),
 default = outcome(() -> infer(rounded, [:X])))
```

```
(invalid = FactorEntryError, log_domain = FactorDomainError, default =
IndeterminatePosteriorError)
```

And an empty query keeps the ordinary unnormalised-mass API, with the log mass available in the diagnostics rather than replacing the returned value.

8 Summary

Log-domain inference is not a different algorithm: `LogVariableElimination` and `LogJunctionTree` run the same elimination order and the same message schedule as their ordinary counterparts (Dechter 1999; Koller and Friedman 2009), with products replaced by sums and sums by log-sum-exp on centered factors. What it buys is the range to carry evidence that is merely rare, and — because a finite log mass and `-Inf` are different numbers, whereas zero and zero are not — the ability to say whether an observation was too small to represent or genuinely impossible. What it does not buy is a bound on the error of a posterior cell. The default backends therefore do not fall back to it: when their Float64 mass cannot tell rare from impossible, they recompute in exact arithmetic, which decides impossibility exactly and rounds each posterior cell once, so `ImpossibleEvidenceError` means probability exactly zero everywhere.

The exact-arithmetic decision path of `InfluenceDiagrams.jl` answers the same concern for decisions rather than posteriors, where the quantity at risk is a sum of signed utilities rather than a product of probabilities.

9 References

Dechter, Rina. 1999. "Bucket Elimination: A Unifying Framework for Reasoning." *Artificial Intelligence* 113 (1–2): 41–85. https://doi.org/10.1016/S0004-3702(99)00059-4.

Koller, Daphne, and Nir Friedman. 2009. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.

Shafer, Glenn R., and Prakash P. Shenoy. 1990. "Probability Propagation." *Annals of Mathematics and Artificial Intelligence* 2: 327–51. https://doi.org/10.1007/BF01531015.